

KULIAH PRAKTEK KOMPUTASI AWAN

MODUL 4.1

Deploy Aplikasi Backend Kas RW

Dari Kode Lokal ke GitHub

Oleh : Dr. Andrew B. Osmond

Mata Kuliah	Praktikum Cloud Computing
Topik	Deployment Aplikasi & Version Control
Studi Kasus	Aplikasi Pencatatan Kas RW
Perangkat	Windows 11 / macOS, VirtualBox, Vagrant, Python, GitHub
Estimasi Waktu	90 - 120 menit

Pada modul ini, mahasiswa akan membangun API backend aplikasi Kas RW menggunakan FastAPI dan Python, menghubungkannya ke database MySQL yang berjalan di vm-database, dan menyimpan seluruh kode ke repositori GitHub. Ini adalah fondasi deployment berbasis version control yang akan digunakan di modul-modul berikutnya.

A. Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- * Membuat repositori kosong di GitHub sebagai remote repository
- * Melakukan git clone untuk menyalin repositori ke komputer lokal
- * Membangun REST API sederhana menggunakan FastAPI dan Python
- * Menghubungkan FastAPI ke MySQL menggunakan mysql-connector-python
- * Menggunakan file .env untuk konfigurasi database yang aman dan portabel
- * Melakukan git add, git commit, dan git push untuk mendorong kode ke GitHub

B. Latar Belakang

Pada Modul 1 dan 2, infrastruktur tiga VM telah berhasil dibangun dan dikonfigurasi otomatis menggunakan Ansible. Namun, server-server tersebut masih kosong -- belum ada aplikasi yang berjalan di atasnya.

Modul ini memulai pembangunan aplikasi Kas RW yang sesungguhnya. Kita akan membuat lapisan backend: sebuah API yang menerima permintaan dari frontend, memproses logika bisnis, dan menyimpan atau mengambil data dari database MySQL di vm-database.

Alur kerja yang kita gunakan mengikuti praktik industri modern: kode dibuat di komputer lokal, disimpan di GitHub sebagai pusat kolaborasi, lalu di-deploy ke server menggunakan git clone. Dengan pendekatan ini, setiap perubahan kode dapat dilacak, dibagikan, dan di-deploy secara konsisten.

Arsitektur yang Kita Bangun

Frontend (vm-frontend) <-> Backend/API (vm-backend) <-> Database (vm-database)

Modul ini fokus pada bagian Backend/API.
Frontend akan dibahas di modul berikutnya.

C. Prasyarat

- * Modul 1 dan 2 sudah diselesaikan: vm-database dan vm-backend sudah berjalan
- * MySQL sudah terinstal di vm-database dengan database kasrw
- * Python 3 sudah tersedia di komputer lokal (cek dengan: `python3 --version`)
- * Git sudah terinstal di komputer lokal (cek dengan: `git --version`)
- * Akun GitHub sudah dibuat

D. Membuat Repositori GitHub

Langkah pertama adalah membuat repositori kosong di GitHub sebagai tempat penyimpanan kode.

1. Buka browser dan masuk ke <https://github.com>
2. Klik tombol New (atau tanda + di pojok kanan atas, lalu New repository)
3. Isi form pembuatan repositori:

Field	Nilai
Repository name	kasrw-backend
Visibility	Public
Initialize with README	Jangan dicentang -- biarkan kosong
Add .gitignore	Jangan dicentang
Choose a license	Jangan dicentang

4. Klik Create repository
5. Salin URL repositori yang muncul, formatnya: `https://github.com/<username>/kasrw-backend.git`

Mengapa Repositori Harus Kosong?

Jika repositori diinisialisasi dengan README atau .gitignore, GitHub akan membuat commit pertama di remote.

Saat kita mencoba git push dari lokal, Git akan menolak karena ada divergensi history. Dengan repositori kosong, push pertama dari lokal berjalan tanpa konflik.

E. Clone Repositori ke Komputer Lokal

Setelah repositori dibuat di GitHub, clone ke komputer lokal untuk mulai mengisi kode.

macOS / Linux -- Terminal

```
cd ~
git clone https://github.com/<username>/kasrw-backend.git
cd kasrw-backend
```

Windows 11 -- PowerShell

```
cd $env:USERPROFILE
git clone https://github.com/<username>/kasrw-backend.git
cd kasrw-backend
```

Ganti <username> dengan username GitHub Anda. Setelah selesai, folder kasrw-backend akan muncul -- kosong, siap diisi kode.

git clone vs git pull

git clone : digunakan pertama kali, menyalin repositori dari remote ke lokal.
git pull : digunakan setelah clone, mengambil perubahan terbaru dari remote.

Karena ini pertama kali, kita pakai git clone.

F. Membuat Struktur Project Backend

F.1. Struktur Folder

Berikut adalah file-file yang akan dibuat di dalam folder kasrw-backend:

```
kasrw-backend/
├── main.py           # Kode utama FastAPI
├── requirements.txt  # Daftar dependencies Python
├── .env.example      # Template konfigurasi (aman di-commit)
└── .gitignore        # File/folder yang tidak masuk ke Git
```

Catatan: File .env

File .env berisi password dan konfigurasi sensitif -- tidak boleh di-commit ke GitHub.
.env.example adalah template kosong yang aman untuk dibagikan ke repositori.
Setiap developer/server yang memakai kode ini akan membuat .env sendiri berdasarkan .env.example.

F.2. File requirements.txt

Buat file requirements.txt di dalam folder kasrw-backend dengan isi berikut:

```
fastapi
uvicorn
```

```
mysql-connector-python
python-dotenv
```

F.3. File .env.example dan .env

Buat file .env.example sebagai template konfigurasi:

```
DB_HOST=192.168.56.11
DB_PORT=3306
DB_NAME=kasrw
DB_USER=kasrw_user
DB_PASSWORD=
```

Salin menjadi .env, lalu isi nilai yang sesuai:

macOS / Linux -- Terminal

```
cp .env.example .env
nano .env      # isi DB_PASSWORD sesuai konfigurasi MySQL Anda
```

Windows 11 -- PowerShell

```
copy .env.example .env
notepad .env      # isi DB_PASSWORD sesuai konfigurasi MySQL Anda
```

F.4. File .gitignore

Buat file .gitignore agar file sensitif tidak masuk ke repositori GitHub:

```
.env
__pycache__/
*.pyc
*.pyo
```

F.5. File main.py

Buat file main.py sebagai kode utama aplikasi FastAPI:

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import Optional
from datetime import date
import mysql.connector
import os
from dotenv import load_dotenv

load_dotenv()
```

```
app = FastAPI(title="Kas RW API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

def get_db():
    return mysql.connector.connect(
        host=os.getenv("DB_HOST"),
        port=int(os.getenv("DB_PORT", 3306)),
        database=os.getenv("DB_NAME"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
    )

class TransaksiBase(BaseModel):
    tanggal: date
    keterangan: str
    jenis: str      # 'pemasukan' atau 'pengeluaran'
    jumlah: float

class TransaksiUpdate(BaseModel):
    tanggal: Optional[date] = None
    keterangan: Optional[str] = None
    jenis: Optional[str] = None
    jumlah: Optional[float] = None

@app.get("/")
def root():
    return {"message": "Kas RW API berjalan"}

@app.get("/transaksi")
def get_all():
    db = get_db()
    cursor = db.cursor(dictionary=True)
    cursor.execute("SELECT * FROM transaksi ORDER BY tanggal DESC")
    rows = cursor.fetchall()
    db.close()
    return rows

@app.get("/transaksi/{id}")
def get_one(id: int):
    db = get_db()
    cursor = db.cursor(dictionary=True)
    cursor.execute("SELECT * FROM transaksi WHERE id = %s", (id,))
    row = cursor.fetchone()
    db.close()
    if not row:
        raise HTTPException(status_code=404, detail="Transaksi tidak ditemukan")
    return row

@app.post("/transaksi", status_code=201)
def create(data: TransaksiBase):
    db = get_db()
    cursor = db.cursor()
    cursor.execute(
```

```
        "INSERT INTO transaksi (tanggal, keterangan, jenis, jumlah) VALUES (%s,
%s, %s, %s)",
        (data.tanggal, data.keterangan, data.jenis, data.jumlah),
    )
    db.commit()
    new_id = cursor.lastrowid
    db.close()
    return {"id": new_id, "message": "Transaksi berhasil ditambahkan"}

@app.put("/transaksi/{id}")
def update(id: int, data: TransaksiUpdate):
    db = get_db()
    cursor = db.cursor(dictionary=True)
    cursor.execute("SELECT * FROM transaksi WHERE id = %s", (id,))
    row = cursor.fetchone()
    if not row:
        db.close()
        raise HTTPException(status_code=404, detail="Transaksi tidak ditemukan")
    updated = {**row, **{k: v for k, v in data.dict().items() if v is not None}}
    cursor.execute(
        "UPDATE transaksi SET tanggal=%s, keterangan=%s, jenis=%s, jumlah=%s
WHERE id=%s",
        (updated["tanggal"], updated["keterangan"], updated["jenis"],
updated["jumlah"], id),
    )
    db.commit()
    db.close()
    return {"message": "Transaksi berhasil diperbarui"}

@app.delete("/transaksi/{id}")
def delete(id: int):
    db = get_db()
    cursor = db.cursor()
    cursor.execute("DELETE FROM transaksi WHERE id = %s", (id,))
    db.commit()
    affected = cursor.rowcount
    db.close()
    if affected == 0:
        raise HTTPException(status_code=404, detail="Transaksi tidak ditemukan")
    return {"message": "Transaksi berhasil dihapus"}
```

Penjelasan Komponen main.py

load_dotenv() : membaca file .env dan memasukkan nilainya ke environment variables.
CORSMiddleware : mengizinkan frontend di domain lain memanggil API ini.
get_db() : membuat koneksi baru ke MySQL setiap kali dipanggil.
int(os.getenv("DB_PORT")) : os.getenv selalu mengembalikan string; port MySQL harus integer.
TransaksiBase : model validasi input untuk POST -- semua field wajib diisi.
TransaksiUpdate : model validasi input untuk PUT -- semua field opsional.

G. Membuat Tabel di vm-database

Sebelum API dapat menyimpan data, tabel transaksi harus dibuat terlebih dahulu di MySQL.

Dari terminal/PowerShell di komputer lokal, masuk ke vm-database:

```
vagrant ssh vm-database
```

Masuk ke konsol MySQL:

```
sudo mysql
```

Jalankan query berikut untuk membuat tabel:

```
USE kasrw;

CREATE TABLE IF NOT EXISTS transaksi (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  tanggal    DATE NOT NULL,
  keterangan  VARCHAR(255) NOT NULL,
  jenis       ENUM('pemasukan', 'pengeluaran') NOT NULL,
  jumlah     DECIMAL(15, 2) NOT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DESCRIBE transaksi;
EXIT;
```

Perintah DESCRIBE transaksi menampilkan struktur tabel yang baru dibuat. Verifikasi bahwa semua kolom muncul sebelum melanjutkan. Keluar dari vm-database:

```
exit
```

Catatan: ENUM pada Kolom jenis

ENUM('pemasukan', 'pengeluaran') berarti MySQL hanya menerima tepat dua nilai tersebut. Jika API mengirim nilai lain (misalnya 'income' atau '3 liter'), MySQL akan menolak dengan error. Ini adalah validasi di level database -- lapisan pertahanan setelah validasi di FastAPI.

H. Install Dependencies dan Uji Coba Lokal

H.1. Install Dependencies

Di terminal/PowerShell, pastikan berada di folder kasrw-backend, lalu install semua dependencies:

```
python3 -m pip install -r requirements.txt
```

Jika pip atau pip3 Tidak Ditemukan

Gunakan python3 -m pip sebagai pengganti pip atau pip3.

Ini terjadi karena pip belum terdaftar di PATH sistem, tapi masih dapat diakses via modul Python.

H.2. Jalankan Server

Jalankan server FastAPI menggunakan Uvicorn:

```
python3 -m uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Jika berhasil, terminal akan menampilkan:

```
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:      Started reloader process
INFO:      Application startup complete.
```

H.3. Uji API via Swagger UI

FastAPI menyediakan dokumentasi dan antarmuka pengujian otomatis. Buka browser dan akses:

```
http://127.0.0.1:8000/docs
```

Tampilan Swagger UI akan muncul dengan daftar semua endpoint. Lakukan pengujian berikut:

- * GET / -- klik Try it out -> Execute. Respons harus: {"message": "Kas RW API berjalan"}
- * POST /transaksi -- isi body JSON berikut, lalu klik Execute:

```
{
  "tanggal": "2025-01-15",
  "keterangan": "Iuran warga RT 01",
  "jenis": "pemasukan",
  "jumlah": 50000
}
```

Respons yang diharapkan (HTTP 201):

```
{
  "id": 1,
  "message": "Transaksi berhasil ditambahkan"
}
```

- * GET /transaksi -- verifikasi bahwa data yang baru dimasukkan muncul dalam daftar

I. Push ke GitHub

Setelah semua file dibuat dan API berhasil diuji, dorong kode ke GitHub.

Dari terminal/PowerShell di dalam folder kasrw-backend, cek terlebih dahulu file yang akan masuk:

```
git status
```

Pastikan file .env tidak muncul dalam daftar (sudah diabaikan oleh .gitignore). Lalu commit dan push:

```
git add .
git commit -m "feat: initial FastAPI backend for Kas RW"
git branch -M main
git push -u origin main
```

Buka GitHub di browser dan verifikasi bahwa semua file muncul di repositori kasrw-backend. File .env seharusnya tidak ada di sana.

Rangkuman Perintah Git yang Digunakan

git clone	: menyalin repositori remote ke lokal (sekali di awal)
git status	: melihat file yang berubah dan statusnya
git add .	: menandai semua perubahan untuk di-commit
git commit	: menyimpan snapshot perubahan dengan pesan deskriptif
git branch -M main	: memastikan branch utama bernama main
git push	: mendorong commit lokal ke GitHub

J. Troubleshooting

pip / pip3 not found

Gunakan python3 -m pip sebagai pengganti. Contoh: python3 -m pip install -r requirements.txt

uvicorn: command not found

Sama seperti pip -- jalankan dengan python3 -m uvicorn main:app --host 0.0.0.0 --port 8000 --reload

500 Internal Server Error saat POST /transaksi

Kemungkinan penyebab:

- * Nilai jenis bukan 'pemasukan' atau 'pengeluaran' -- MySQL menolak nilai di luar ENUM
- * Koneksi ke vm-database gagal -- pastikan VM berjalan (vagrant status) dan IP di .env sudah benar
- * DB_PORT tidak dikonversi ke int -- pastikan kode menggunakan `int(os.getenv("DB_PORT", 3306))`

ImportError: cannot import name 'FastAPI' from 'fastAPI'

Nama modul Python bersifat case-sensitive. Pastikan penulisan import menggunakan huruf kecil semua:

```
from fastapi import FastAPI, HTTPException    # huruf kecil semua: 'fastapi'
```

Access denied for user saat koneksi MySQL

Pastikan user MySQL yang dikonfigurasi di .env memiliki akses dari IP komputer lokal. Di vm-database:

```
sudo mysql
SELECT user, host FROM mysql.user WHERE user = 'kasrw_user';
EXIT;
```

Kolom host menunjukkan dari IP mana user tersebut diizinkan login.

git push ditolak (error: failed to push)

Pastikan repositori di GitHub benar-benar kosong saat pertama kali push. Jika ada README yang terbuat otomatis, hapus repositori dan buat ulang tanpa mencentang opsi inisialisasi apapun.

K. Kesimpulan

Pada modul ini, kita telah berhasil membangun API backend Kas RW yang berfungsi penuh, menghubungkannya ke database MySQL, dan menyimpan seluruh kode ke GitHub menggunakan workflow version control yang benar.

Yang Sudah Dikerjakan	Yang Akan Datang
Repositori GitHub dibuat dan di-clone ke lokal REST API FastAPI lengkap (GET, POST, PUT, DELETE) Koneksi ke MySQL berhasil menggunakan .env API diuji via Swagger UI Kode di-push ke GitHub	git clone ke ec2-backend di AWS Membungkus backend ke dalam Docker image Membangun frontend Kas RW